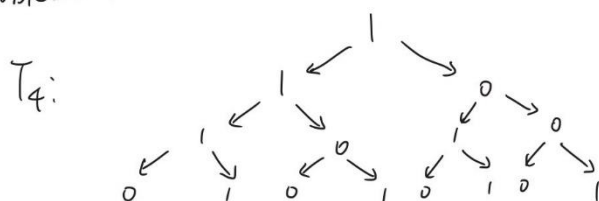


Problem 1



S_4 : 0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1

1-2

$$S_n = (a_1, a_2, \dots, a_{\bar{n}}, \dots, a_{2^n - \bar{n}}, \dots, a_{2^n - 2}, a_{2^n - 1})$$

$$\text{let } S_{n-1} = (a_1, a_2, \dots, a_{\bar{n}}, \dots, a_{2^{n-1} - 1})$$

$\Rightarrow S_n[\bar{x}] = S_{n-1}[\bar{x}]$ (since this equation is symmetric w.r.t $\bar{x}$ and $2^n - \bar{x}$, we can assume that $\bar{x}$ is in the left half of the sequence.)

For the right subtree string of S_n , inverting a tree flips the value of every node, so index j becomes $1 - S_{n-1}[j]$,

and mirroring a tree reverses its in-order traversal. Since the length of S_{n-1} is $2^{n-1} - 1$, the element at index j in R_n corresponds to the element at index $2^{n-1} - j$ in S_{n-1}

$$\Rightarrow R_n[j] = 1 - S_{n-1}[2^{n-1} - j]$$

$$S_n[\bar{x}] = S_{n-1}[\bar{x}]$$

$$S_n[2^n - \bar{x}] = R_n[2^{n-1} - \bar{x}] = 1 - S_{n-1}[2^{n-1} - (2^{n-1} - \bar{x})]$$

$$= 1 - S_{n-1}[\bar{x}]$$

$$\Rightarrow S_n[\bar{x}] + S_n[2^n - \bar{x}] = 1$$

Noted that in this assignment, the $\wedge$ sign stands for power; bitwise OR will be written as XOR.

1-3

```
1  findValue(i, n)
2      if n > 1 and i == 2 ^ (n - 1)
3          return 1
4      else if i == 1
5          return 0
6      else if(i > 2 ^ (n - 1))
7          return 1 - findValue(2 ^ n - i, n - 1)
8      else
9          return findValue(i, n - 1)
```

the first base case is hit when $n = 1$, which is leaf nodes of the tree T_1 . Because the algorithm mirrors all the node on the right to its left counterpart, so every node on the leaf will eventually be reduced to the leftmost node, which yields to a value of 0.

The second base case is hit when $n > 1$ and i is the root of the tree (i.e. $i == 2^{(n-1)}$), since the root of the left subtrees will always be 1 based on the construction rule of the tree.

For case when i is in the right subtree ($i > 2^{(n-1)}$), where $2^{(n-1)}$ is the root of the tree with depth n), we can calculate its value by subtracting the value of its symmetry node on the left by 1. So we return $(1 - \text{findValue}(2^n - i, n - 1))$, setting $2^n - i$ mirrors the node and trace one layer below to depth $n - 1$ at this step. This is the property derived from [question 1-2](#).

For the case when i is on the left subtree ($i < 2^{(n-1)}$), we call $\text{findValue}(i, n - 1)$, we trace one layer below.

The worst-case scenario happens when the target node is on the leaf, but for whichever side the node is on, the algorithm will always search one layer below and do n repetitions. So the worst case time complexity is $O(n)$.

1-4

For T_n , $W(n) = 1 + \text{sum of left subtree}$
 $+ \text{sum of right subtree}$

$$= 1 + W(n-1) + \sum_{\tilde{i}=2^{n-1}+1}^k S_n[\tilde{i}]$$

As proved in question 1-2, $S_n[\tilde{i}] = 1 - S_n[2^n - \tilde{i}]$

Case $k > 2^{n-1}$ (node k is on the right subtree)

$$\begin{aligned} \Rightarrow P(n, k) &= 1 + W(n-1) + \sum_{\tilde{i}=2^{n-1}+1}^{2^n-k} (1 - S_{n-1}[\tilde{i}]) \\ &= 1 + W(n-1) + \sum_{\tilde{i}=2^{n-1}+1}^{2^n-k} 1 - \sum_{\tilde{i}=2^{n-1}+1}^{2^n-k} S_{n-1}[\tilde{i}] \\ &= W(n-1) + \sum_{\tilde{i}=2^{n-1}+1}^{2^n-k} 1 - (W(n-1) - P(n-1, 2^n-k-1)) \\ &= (2^n-k-1) + 1 + P(n-1, 2^n-k-1) \\ &\quad \left(\sum_{\tilde{i}=2^{n-1}+1}^{2^n-k} 1 = \sum_{\tilde{i}=2^{n-1}+1}^k 1 = k - 2^{n-1} \right) \end{aligned}$$

The reason why $k^* = 2^n - k - 1$ in $P(n-1, 2^n-k-1)$

is because for $2^{n-1} < k < 2^n - 1$, the last

$(2^n - 1 - k)$ nodes will be left out. After mirroring them to the left, we should subtract the first $(2^n - 1 - k)$ nodes from $W(n-1)$.

Continue on the next page.

Case $k < 2^{n-1}$ (node k in on the left subtree)

$$P(n, k) = P(n-1, k)$$

Since k is in the left subtree, the k^{th} node in T_n is essentially the k^{th} node in T_{n-1}

Case $k = 2^{n-1}$

$$P(n, k) = P(n-1, 2^{n-1} - 1) + 1$$

except for counting the sum of the left subtree, we still have to add the root, whose value is 1.

1-5

```
1  findTotal(n, k)
2      if n == 1 or k == 0
3          return 0
4      else
5          if k < 2 ^ (n - 1) // node k is on the left subtree
6              return findTotal(n - 1, k)
7          else if k > 2 ^ (n - 1) // node k is on the right subtree
8              return k - 2 ^ (n - 1) + findTotal(n - 1, 2 ^ n - k - 1) + 1
9          else
10             return findTotal(n - 1, 2 ^ (n - 1) - 1) + 1
```

the base case is hit when $n = 1$, which only includes one single node with the value of 0, or $k = 0$, indicating that there is no node to calculate so the value is also 0.

For case when the target node k is on the left subtree and right subtree, the return values are the same as derived in [question 1-4](#), so the explanation will be the same.

For case when the target node k is the root of the tree, we recursively sum the entire left subtree and add 1 for the root itself. Because the algorithm evaluates one condition and makes exactly one recursive call per depth level n , the time complexity is $O(n)$

1-6

```
1  findLCValue(n, k1, k2)
2      inv = 0
3      root = 2 ^ (n - 1)
4      while true
5          if k1 < root and k2 < root
6              root = root - 2 ^ (n - 2)
7              n = n - 1
8
9          else if k1 > root and k2 > root
10             root = root + 2 ^ (n - 2)
11             n = n - 1
12             inv = 1 - inv
13
14         else
15             if n == 1
16                 value = 0
17             else
18                 value = 1
19         return inv XOR value
```

```
22  subpathXOR(k, n, inv)
23      if n == 1
24          value = 0
25      else
26          value = 1
27
28      if inv == 1
29          value = 1 - value
30
31      if k == 2 ^ (n - 1)
32          return value
33      else if k < 2 ^ (n - 1)
34          return value XOR subpathXOR(k, n - 1, inv)
35      else
36          return value XOR subpathXOR(2 ^ n - k, n - 1, 1 - inv)
```

```
38  pathXOR(n, k1, k2)
39      return subpathXOR(k1, n, 0) XOR subpathXOR(k2, n, 0) XOR findLCValue(n, k1, k2)
```

The first step is to find the value of LCA. By using a while loop, we start from the root of the tree, if both nodes are on the left subtree, we modify our target root to the root of the left subtree; if both nodes are on the right subtree, we modify our target root to the root of the right subtree; if the two nodes are on different side, then the root we are current at is the LCA. We will then return the value (0 or 1) of the LCA. The inv variable indicates the inversion of the node. Inv will switch every time when the algorithm traces down the right subtree.

The second step is to find the subpathXOR for the left and right subpath respectively. The variable “value” stands for the current XOR result on the path. We initialize the value of the LCA. If it is on the leaf, we set “value” to 0, 1 otherwise. Additionally, if $inv = 1$, indicating that the LCA is inverted an odd number of times, we should change “value” to 1 if it was 0 and vice versa. Then we do the XOR operation along the path from the LCA to k_1 and k_2 , respectively, by recursion. If the target node (k_1 or k_2) happens to be the LCA, we return “value” itself; if the target is on the left subtree, we return the result of “value” XOR “the result of the path from the target node to the node that is one layer below”; the same logic applies when the target node is on the right subtree. The only difference is that we change k to $2^n - k$.

In the last step we simply return the XOR of the result of the left and right subtree, and XOR it with the LCA value. Since it will be calculated twice when we separate the path into left and right subpath, we XOR the LCA value again to avoid the double calculation.

Because finding the LCA and tracing the two subpaths only requires traversing straight down the tree, the operations are bounded by the maximum depth n , guaranteeing an $O(n)$ time complexity.

2-1

```
1  trainSort(S1, S2, S3)
2      stackTrackingArr = createArray(len = n, initialValue = 1)
3      for i from n to 1
4          if stackTrackingArr[i] == 1
5              while S1.peek() != i
6                  S1.pop() -> S2.push()
7                  stackTrackingArr[S2.peek()] = 2
8                  S1.pop() -> S3.push()
9          else
10             while S2.peek() != i
11                 S2.pop() -> S1.push()
12                 stackTrackingArr[S1.peek()] = 1
13             S2.pop() -> S3.push()
```

First, we create a “stackTrackingArr” array to track the stack where each train is currently on. We initialize each element to 1 since all trains are initially on track S1.

Then we run a for loop for n iterations. In each iteration, we search train i from the track that i is currently on. Since we cannot move train out of T3, we can only move trains on T1 and T2 before we find our target train. If the top train of that specific track is not our target i, we move that train to the other track and update its “stackTrackingArr” value. We repeat this step until we find the ith train, which will then be moved into T3.

For the inner while loop, the worst-case scenario will be the ith train is placed in the bottom. In this case, the while loop will have to run $i - 1$ iteration. Therefore, in the worst-case scenario, the total steps will be

$$(n - 1) + (n - 2) + \dots + 2 + 1 = (n^2 - n) / 2$$

So the overall time complexity will be $O(n^2)$.

```

1  processingQueue(processList, Q0, Q1)
2      completionTimeArray = createArray(len = n)
3      for i from 1 to n
4          while not isEmpty(Q0) and processList[i].t >= Q0.peek()
5              Q0.dequeue()
6          while not isEmpty(Q1) and processList[i].t >= Q1.peek()
7              Q1.dequeue()
8
9          activeQueue = Q0
10         if size(Q1) < size(Q0)
11             activeQueue = Q1
12
13         completionTime = processList[i].c
14         if not isEmpty(activeQueue)
15             completionTime += activeQueue.getLastElement()
16         else
17             completionTime += processList[i].t
18         completionTimeArray[i] = completionTime
19         activeQueue.enqueue(completionTime)
20
21
22     return completionTimeArray

```

Noted that we assume that “process” is pre-structured and contains two numerics: the arrival time t and the execution time c . so the “processList” is an array that stores all the processes, and the two processors are $Q0$ and $Q1$, since the logic of their workflow act like queues.

First, we create a “completionTimeArray” which will store the completion time of each process.

Next, we run a for loop for n iterations, since the arrival time t is strictly increasing, we use a while loop to check if there’s any process done earlier than the time t for both queues. After the check and removal process for expired processes, we will check size of both queue and push the new process to the queue that has a smaller size. We first set up and initialize the variable “completionTime” to be the execution time c of the process, and add it with the completion time of the last element in the queue since that will be the time when the i th process can start being processed. It is worth noting that we will keep the algorithm will keep track of the tail of the queue, so the

getLastElement function will spend $O(1)$ time to get the value. If the queue is empty, we will add “completionTime” with the arrival time t of the process. We will push completionTime afterwards.

Since a process will be enqueued and dequeued for at most once, and that all the other operations are done in constant time, the total time complexity will satisfy $\sum T(i) = O(n)$.

For the space complexity, since we only create a “completionTimeArray” that has a length of n and other variables owns constant spaces, the total space complexity is $O(n)$.

```

1  monotonicSort(array A, len n)
2      indexArray = createArray(len = n)
3      top = 0
4      for i from 1 to n
5          while top > 0 and A[i] >= A[indexArray[top]]
6              top--
7              top++
8              indexArray[top] = i
9
10     return indexArray

```

Note that the original array is named A , and the `indexArray` is essentially the memory sequence S specified in the homework instruction.

First, we create an array to store the index and set the `top` pointer to 0 (the array is 1-indexed). As we iterate through A with an index i , we pop elements from the top of the stack as long as the current element $A[i]$ is greater than or equal to the element at the top of the stack, so the array will be monotonic decreasing.

The outer for loop is going to run n times, so the operation `top++` will be executed n times, leading to n times of pushes in total.

The inner while loop executes the `top--` operation. Since every element will only be popped for at most once, this means that throughout the for loop, the while loop will only be executed at most n times in total.

With n pushes and n pops at most and all the other operations taking $O(1)$ time, the total number of operations is bounded by $2n$. Hence, the total time spent satisfies $\sum T(i) = O(n)$.

```

1  maxScore(array A, indexArray, left, right, top)
2      leftPointer = 1, rightPointer = top
3      bestTarget = indexArray[top]
4      while leftPointer <= rightPointer
5          mid = (leftPointer + rightPointer) / 2
6          if left <= indexArray[mid]
7              bestTarget = indexArray[mid]
8              rightPointer = mid - 1
9
10         else
11             leftPointer = mid + 1
12
13     return A[bestTarget]

```

```

15  findMaxScores(array A, len n, queries)
16      k = queries.len // assume "queries" is an array with a length of k that contains k queries
17      indexArray = createArray(len = n)
18      result = createArray(len = k)
19      top = 0
20      j = 1
21      for i from 1 to n
22          while top > 0 and A[i] >= A[indexArray[top]]
23              top--
24              top++
25              indexArray[top] = i
26          while j <= k && i == queries[j].right
27              result[j] = maxScore(A, indexArray, queries[j].left, queries[j].right, top)
28              j++
29
30  return result

```

Noted that the algorithm assumes that “query” is a pre-defined structure which contains left and right bound; q and k are interchangeable since they both stand for the number of queries, or the length of the queries array; the indexArray is essentially the memory sequence S

First, we create a helper function to search for the max value for one single query. This function will take the input array (named A), the sorted indexArray at the current step, the left and right bound, and the current top pointer of the indexArray.

Then, we use binary search to search for the first index in the indexArray that is greater than the left bound. This will work because the indexArray has the property that the indices will be monotonic increasing since we push the indices orderly. The binary search approach make the worst-case time complexity $\log(n)$.

The main `findMaxScores` function takes the input array `A`, the length of the array, and the `queries` array that stores all q queries.

We first create the `result` array that stores the maximum value for each query, so it will have a length of k .

The for loop (line 21) will push the current index into the `indexArray` after the operation in the while loop (line 22), which pops all the indices whose value is less than the value of the current index. With n pushes and n pops at most and all the other operations taking $O(1)$ time, the total number of operations is bounded by $2n$. Hence, the total time spent in this step satisfies $O(n)$.

When our current index i matches the right bound of a query, we calculate the answer for that query. We call the helper function `maxScore` to get the maximum value for the current query and store it into the result array. We use `j` to track the tail of the array. Since the right bound is increasing, looking for the maximum value before reading the next indices the current indices being overwritten.

In total, the `maxScore` function will be called k times, resulting in a total time complexity of $k * \log(n)$.

Overall, with the push and pop approach as well as the binary search, the total time complexity will be $O(n) + O(k * \log(n)) = O(n + k \log(n))$.